

AICE Evolution v2

多タスク横断進化探索による次世代プログラミング言語の予測

aice-evolution-v2 / 実験レポート

2026 年 5 月

Abstract

プログラミング言語の進化を遺伝的アルゴリズム (GA) で表現する従来手法は、遺伝子・突然変異・交叉を手で記述する負担が大きい。本研究では (1) 遺伝子スキーマを 1 度書けば変異と交叉が機械派生される MAP-Elites 探索、(2) 複数のタスクドメイン横断で進化方向ベクトルを集計する手法、(3) 同一仕様から Python パイプラインと **AIPL** (Actor-based Intelligent Parallel Language; 旧称 ABCL/c+) アクター並列実装の双方を自動生成するコード生成器、を組み合わせ、6 サンプル問題 × 5 実験で“次世代プログラミング言語の方向”を機械抽出した。結果として型安全性は 5/5 ドメインで強化される (sign-agree 100%)、雑な並行モデルは 4/5 ドメインで淘汰される (sign-agree 80%) という 2 軸が普遍的進化圧力として抽出された。Pareto-top の多数決から得られる consensus genome は `capability` 効果系 + `symbol_owned` 状態 + `high` 型安全性 + `linear` 所有権という組み合わせで、最近傍既存言語は Pony、未充足領域は `capability` × `linear` × `dependent types` × `structured concurrency` の交点であることを示した。

1 はじめに

1.1 問題設定

ソフトウェアの進化を表現する際、AICE メタ言語上で GA を用いてアセンブラ → BASIC → C → Java → ParallelOOP のような系譜を辿らせる試みは可能である。しかし従来手法では、

- 遺伝子の具体的な内容 (どの軸でどんな値を取るか)
- 突然変異 (例: Assembly_to_BASIC)
- 交叉 (例: JavaOOP × FunctionalOOP → ParallelOOP)

の全てを手で書く必要があり、自動化の余地が大きい。本研究の出発点となった問いは次の 3 つである:

1. 遺伝子と操作子を自動派生できないか?
2. 進化軌跡を 副産物 として系譜から得られないか?
3. この仕組みで「次世代プログラミング言語はどのようなものが良いか」を機械的に予測できるか?

1.2 採用したアプローチ

- (1) **スキーマ駆動** `gene_schema` に「軸 × 許容値 × coherence 制約」を 1 度書けば、突然変異 (1 軸ランダム書き換え) と交叉 (ユニフォーム交叉) は機械的に派生される。
- (2) **MAP-Elites** 単一の最適解ではなく 各セルの *champion* を保持する Quality-Diversity 探索を採用。系譜を全保存することで、最深 *champion* の軌跡を“発見された進化道筋”として読み出せる。
- (3) **多タスク横断** 同じスキーマで異なる問題ドメイン 5 種を独立に走らせ、各 run の trend ベクトルを横断集計。タスクを超えて方向が一致する軸を 普遍的進化圧力、ばらつく軸を *task-specific* ノイズ として分離した。

2 手法 — 7 段階パイプライン

開発は 7 つの Phase に分けて段階的に行った。各 Phase は前段階の成果物を入力にとり、独立にテスト可能である (表 1)。

Table 1: Phase 構成

#	Phase 名	主成果物	内容
1	スキーマ + IR	schema.json / ga.json	遺伝子軸 / 許容値 / coherence 制約と中間表現
2	MAP-Elites	map_elites.py	セル別 champion + 全系譜トラッキング
3	解析 + ランキング	analysis.py / ranking.py	trend ベクトル / Pareto gap / 多シナリオランキング
4	DSL パーサ	aice_parser.py	.aice v2 → IR への lowering
5	AIPL コード生成	aipl_codegen.py	IR → AIPL オーケストレータ
6	実 LLM 結線	ai_evaluator.py	reviewer / pairwise judge を ai_call 化
7	多タスク横断集計	cross_task.py / batch.py	N runs を集計 → universal direction 抽出

2.1 データの流れ

人間が書く	自動生成
schemas/*.schema.json	→
examples/*.aice (5 ファイル)	→ out/*.ga.json
	→ out/*.abcl (オプション)
	→ out/*.lineage.json
	→ out/*.elite_map.json
	→ out/*.ranking.json
	→ out/*.report.md
	→ out/final_prediction.cross_task.{md,json}

3 サンプル問題

抽象タスクとして 6 つを用意した (表 2)。各タスクには“理想プロファイル” (gene 値の組み合わせ) を割り当て、評価器がそれとの軸一致率をスコアにする。

Table 2: サンプル問題と理想プロファイル

タスク	内容	理想プロファイル抜粋
TrafficLight	信号機 tick (Red → Yellow → Green) を並行 tick 要求下で安全に処理	ParallelOOP, actor_messages, type_safety=high
BankAccount	残高保持 + deposit/withdraw/transfer の原子性	Java_OOP, threads_locks, type_safety=high, gc
Philosophers	食事哲学者 + デッドロック回避	ParallelOOP, actor_messages, borrow_check
WebService	REST + 認可 + 構造化並行	FunctionalOOP, structured, algebraic_effects, gc
DataPipeline	ストリーミング ETL + 純粋関数ステージ	Functional, ADT, csp_channels, monadic
Compiler	AST 変換 + 型主導書き換え	Functional, ADT, type_safety=dependent

4 実装 — .aice 仕様と変換

4.1 .aice v2 — 人間が書く DSL

リスト 1 は最終デモで使った .aice の例である (全 5 ファイルが同形式)。stages / mutations / crossovers を書く必要がない点が従来手法との最大の差で、人間の負担は約 1.5 KB に収まる。

Listing 1: examples/NextLanguagePrediction.aice (抜粋)

```

aice NextLanguagePrediction {
  task = "プログラミング・パラダイムの進化軌跡を発見し、次世代言語の候補を提示する。";
  gene_schema = "schemas/programming_paradigm.schema.json";

  evaluation_tasks {
    task TrafficLight { spec = "..."; }
    task BankAccount { spec = "..."; }
    task Philosophers { spec = "..."; }
  }

  search {
    algorithm = "map_elites";
    cell_axes = "paradigm, concurrency_model, type_safety";
    generations = 30;
    seed_count = 8;
    open_axes = true;
  }

  meta_fitness {
    trend_alignment = 0.20; novelty = 0.15;
    frontier_coverage = 0.20; evolvability = 0.15;
    cross_task_generality = 0.20; implementability = 0.10;
  }

  ranking {
    scenarios = ["conservative", "innovative", "general_purpose"];
    top_k = 3;
  }
  reviewers { ... }
}

```

4.2 パーサと中間表現 (.ga.json)

aice_parser.py がトークナイズして AST を構築し、lowering 段階で operators / scenario_weights のデフォルトを充填して .ga.json (約 2.8 KB) に書き出す。

4.3 AIPL コード生成

aip1_codegen.py は .ga.json を入力に、9 個のアクタークラスを持つ AIPL プログラム (約 19 KB / 580 行) を生成する (表 3)。

Table 3: 生成 AIPL プログラムのアクター構成

アクター	役割
Util	CSV / 遺伝子文字列ヘルパ + parse_score (LLM 応答 → float)
DigitOps	parse_score 補助 (peer actor; self 再入回避)
Generator	seed / mutate / cross 操作
Reviewer	1 reviewer = 1 actor; ai_call_with_system 経由でスコアリング
Evaluator	3 reviewer に future で fan-out → await で集約
Worker	compute_cell / compute_score (Coordinator の代理計算)
EliteMap	cell → champion テーブル (parallel arrays)
Lineage	全個体を append-only 記録 → JSON dump
TaskProfiles	各タスクの理想プロファイルを返す
Coordinator	Phase 1 (seed) → Phase 2 (進化ループ) を駆動

実装で踏んだ AIPL ランタイムの注意点を表 4 に記録する。これらは生成器の知識として固定化済みである。

Table 4: AIPL コード生成で踏んだ罠

症状	対処
<code>now self.X()</code> で無限デッドロック	同アクター内では再入不可。Worker / DigitOps として peer 化
<code>xs = array_push(xs, v)</code> で <code>xs</code> が <code>None</code> に	Python ランタイムは in-place + <code>None</code> 返却。戻り値破棄
<code>float(x)</code> がパースエラー	<code>float</code> は型キーワード予約。 <code>(x + 0.0)</code> で coerce
<code>a && b</code> がパースエラー	論理 AND/OR 無し。ネスト if で書く
<code>0.0</code> の <code>stringify</code> が <code>"0."</code>	末尾 <code>"0"</code> を付与して JSON valid に

5 実験

5.1 構成

5 つの `.aice` を用意し、それぞれ異なる task subset を採用した (表 5)。全実験は同一の `rng_seed = 42` で実行し、独立変数は採用タスクと reviewer persona のみである。

Table 5: 実験 5 種の構成

.aice	採用タスク	想定 domain
NextLanguagePrediction	TrafficLight + BankAccount + Philosophers	並行 + 状態機械
WebServiceEvolution	WebService + BankAccount + TrafficLight	リクエスト処理
DataPipelineEvolution	DataPipeline + Compiler + TrafficLight	関数型 + 型
CompilerEvolution	Compiler + DataPipeline + BankAccount	型システム
UltimateNextLanguage	全 6 タスク	総合判定

5.2 Python パイプライン実行結果

各 run の MAP-Elites 結果 (Python 側) を表 6 に示す。UltimateNextLanguage は他より `seed_count=12`, `generations=40` で大きいため個体数が増える。

Table 6: Python パイプライン: 各 run の規模と最高 composite

run	個体	セル	best	best 候補抜粋
NextLanguagePrediction	38	20	0.795	ParallelOOP / actor / type=high
WebServiceEvolution	38	20	0.795	ParallelOOP / actor / type=high
DataPipelineEvolution	38	19	0.837	Functional / structured / type=high
CompilerEvolution	38	19	0.837	Functional / structured / type=high
UltimateNextLanguage	52	30	0.797	ParallelOOP / actor / type=high

5.3 AIPL アクター並列実行結果

Python パイプラインと並行して、生成された 5 つの `.abc1` を `ABCL_AI_PROVIDER=mock` で実行した (表 7)。RNG パスが Python と独立なため個体集合は異なるが、cell 充填数はむしろ多くなり、actor framework が end-to-end で動作することを確認した。

Table 7: AIPL ランタイム実行結果 (.abc1)

run	個体	セル	best	best 候補抜粋
NextLanguagePrediction	38	24	0.417	ParallelOOP + actor_messages
WebServiceEvolution	38	22	0.433	C + structured + type=high
DataPipelineEvolution	38	19	0.417	assembler + type=high
CompilerEvolution	38	19	0.467	C + type=high
UltimateNextLanguage	52	30	0.483	Functional + csp_channels

6 結果 — 多タスク横断集計

6.1 進化方向ベクトルの軸別安定性

5 runs の trend ベクトルを集計し、各軸ごとに mean / std / sign-agree を計算した結果が表 8 である。

Table 8: 軸ごとのタスク横断安定性 (5 runs)

axis	mean	std	+n / -n / 0n	sign-agree
type_safety	+0.42	0.10	5 / 0 / 0	100%
concurrency_model	-0.22	0.23	1 / 4 / 0	80%
ownership_model	-0.08	0.30	3 / 2 / 0	60%

6.2 Universal direction (普遍的進化圧力)

sign-agreement $\geq 75\%$ かつ $|\text{mean}| > 0.05$ を “universal” の閾値とした:

- type_safety: +0.42 (sign-agree 100%, runs 5 $\uparrow$ /0 $\downarrow$)
- concurrency_model: -0.22 (sign-agree 80%, runs 1 $\uparrow$ /4 $\downarrow$)

これらは個別タスクのノイズではなく、5 つの異なる問題ドメインを横断して再現した進化方向 である。

6.3 Consensus genome (Pareto-top の多数決)

各 run の Pareto-1 候補から軸別の最頻値を集計したのが表 9 である。最大の合意は effect_handling=capability (62%) で、type_safety=high と state_representation=symbol_owned がそれに続く。

Table 9: Consensus genome (5 runs の Pareto-top 候補 $N = 34$ からの多数決)

axis	winner value	votes / total
effect_handling	capability	21 / 34 (62%) 最強
type_safety	high	18 / 34 (53%)
state_representation	symbol_owned	18 / 34 (53%)
ownership_model	linear	14 / 34 (41%)
concurrency_model	csp_channels	12 / 34 (35%)
paradigm	BASIC	12 / 34 (35%) (注: セル名のみ)

7 次世代プログラミング言語の予測

機械抽出の結果、推奨される性質は次の 5 軸に集約された:

- `type_safety = high` (もしくは更に `dependent`) 5/5 ドメインで一致
- `effect_handling = capability` 全候補中 62% で支持される最強の合意
- `state_representation = symbol_owned` 単一所有 + 移譲モデル
- `concurrency_model = csp_channels` もしくは `structured` 雑な `threads_locks` は淘汰圧
- `ownership_model = linear` (もしくは `borrow_check`) GC は task-specific でやや弱い

7.1 既存言語との対応

最近傍 **Pony** — `capability + actor + linear types` を満たす。

次点 **Rust** + `capability + algebraic effects` — まだ言語として存在しない。

関数型側 **Scala / Effekt** — `effect tracking + structured concurrency` を満たす。

7.2 Pareto frontier の未充足領域 — 次世代言語の置き場

`capability × linear × dependent types × structured concurrency` を全て満たす言語は実在しない。これが次世代言語が埋めるべき設計空間である。

8 議論 — できたこと / できなかったこと

8.1 達成できたこと

1. 遺伝子・突然変異・交叉を 1 度のスキーマ宣言 から機械派生する仕組みが完成した。
2. MAP-Elites の系譜トラッキングにより、“発見された進化軌跡” を副産物として読み出せる。
3. 同一 `.aice` から Python と AIPL アクターの二重実装を自動生成し、両方で end-to-end が動作した。
4. Phase 7 の多タスク横断集計で、タスク固有のノイズと普遍的進化圧力が定量的に分離 できた。

8.2 未解決の限界

1. サンプルサイズの小ささ — 5 runs では各軸の mean / std の信頼区間が広い。`rng_seed` を変えた追加実行 (Phase 8 候補) が望ましい。
2. モック評価器の限定性 — 本番では `--ai` で実 LLM 化できるが、本実験はコスト都合で deterministic な軸一致率に留めた。
3. 軸の閉鎖性 — `open_axes = true` でも、現状の探索器は新軸を提案しない。LLM 駆動で新しい軸 (例: “differentiable types”) を生成する開放系は Phase 9 候補。
4. 真のパラダイム転換は出ない — ハードウェア・ML・量子計算といった外的圧力は本枠組みではモデル化しない。予測は“既知軸の延長線上”に閉じる。

9 結論

7 段階に分けて構築した `aice-evolution-v2` は、人間が `.aice` を 1.5 KB 程度書くだけで MAP-Elites 探索 → 進化軌跡発見 → Pareto ランキング → Markdown レポート → AIPL アクター並列実装 → 多タスク横断集計までを end-to-end で自動化した。

5 ドメイン横断の集計では、型安全性は普遍的に強化される (100% sign-agreement) という強い結論と、`capability` 効果系 + `linear` 所有 + `symbol_owned` 状態 + `high` 型安全性 という consensus

genome を得た。最近傍既存言語は Pony、未充足領域は `capability × linear × dependent × structured` の交点である。

これは“単一の言語名”を当てる予測ではなく、**満たすべき設計軸の集合と、その不確実性** を返す予測である。本枠組みは出力に重み・シナリオ・信頼区間を保持するため、“保守的予測 / 革新的予測 / 汎用予測”という 3 つの並行ランキングを出すことができ、外的条件次第で複数の解釈が可能である点を誠実に記録できる。

A 再現コマンド

Listing 2: 5 実験の一括実行と最終集計

```
# Python パイプライン (5 runs + 横断集計)
python3 -m src.batch \
    examples/NextLanguagePrediction.aice \
    examples/WebServiceEvolution.aice \
    examples/DataPipelineEvolution.aice \
    examples/CompilerEvolution.aice \
    examples/UltimateNextLanguage.aice \
    -o out --name final_prediction

# AIPL アクター並列実装の生成と実行 (各 .aice ごと)
for ex in NextLanguagePrediction WebServiceEvolution \
    DataPipelineEvolution CompilerEvolution \
    UltimateNextLanguage; do
    python3 -m src.cli examples/$ex.aice -o out --abcl --no-run
    ABCL_AI_PROVIDER=mock python3 ../src/python-aipl/aipl_main.py \
        --timeout 240 out/$ex.abcl
done
```

B ファイル一覧

人間が書く入力

- `schemas/programming_paradigm.schema.json` (1.7 KB)
- `samples/problems.md` (4.5 KB)
- `examples/*.aice` 5 ファイル (1.5–1.9 KB 各)

自動生成出力

- `out/*.ga.json` (各 ~3 KB)
- `out/*.abcl` (各 ~19 KB)
- `out/*.lineage.json` (各 28–44 KB)
- `out/*.elite_map.json` (各 ~8 KB)
- `out/*.ranking.json` (各 ~45 KB)
- `out/*.report.md` (各 ~4.5 KB)
- `out/final_prediction.cross_task.{md,json}` (各 ~2 KB)